

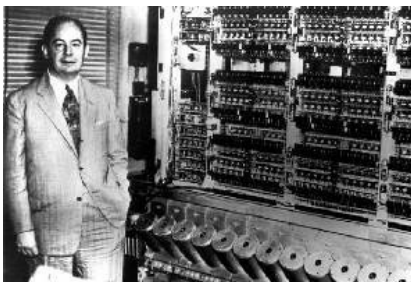
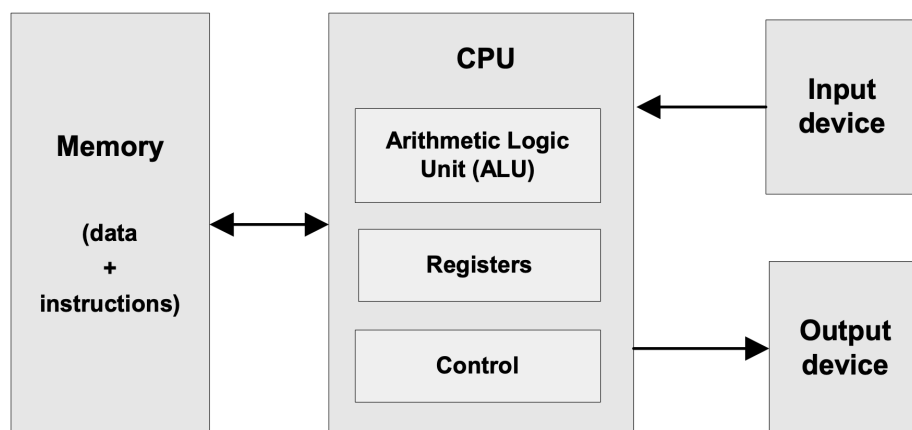
Computer Architecture



Building a Modern Computer From First Principles

1

Von Neumann machine (circa 1940)



John Von Neumann (and others) ... made it possible

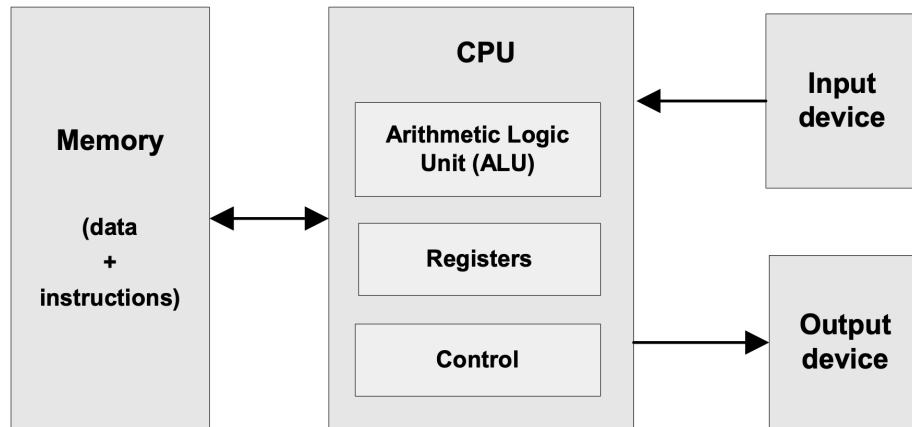


Stored
program
concept!

Andy Grove (and others) ... made it small and fast.

4

Processing logic: fetch-execute cycle



Executing the current instruction involves one or more of the following micro-tasks:

- Have the ALU compute some function $out = f(\text{register values})$
- Write the ALU output to selected registers
- As a side-effect of this computation, figure out which instruction to fetch and execute next.

5

The Hack chip-set and hardware platform

Elementary logic gates	Combinational chips	Sequential chips	Computer Architecture
Nand	HalfAdder	DFF	Memory
Not done	FullAdder	Bit	CPU
And	Add16	Register	Computer
Or	Inc16	RAM8	this lecture
Xor	ALU	RAM64	
Mux	done	RAM512	
Dmux		RAM4K	
Not16		RAM16K	
And16		PC	
Or16			
Mux16		done	
Or8Way			
Mux4Way16			
Mux8Way16			
DMux4Way			
DMux8Way			

6

The Hack computer

A 16-bit Von Neumann platform

The instruction memory and the data memory are physically separate

Screen: 256 rows by 512 columns, black and white

Keyboard: standard

Designed to execute programs written in the Hack machine language

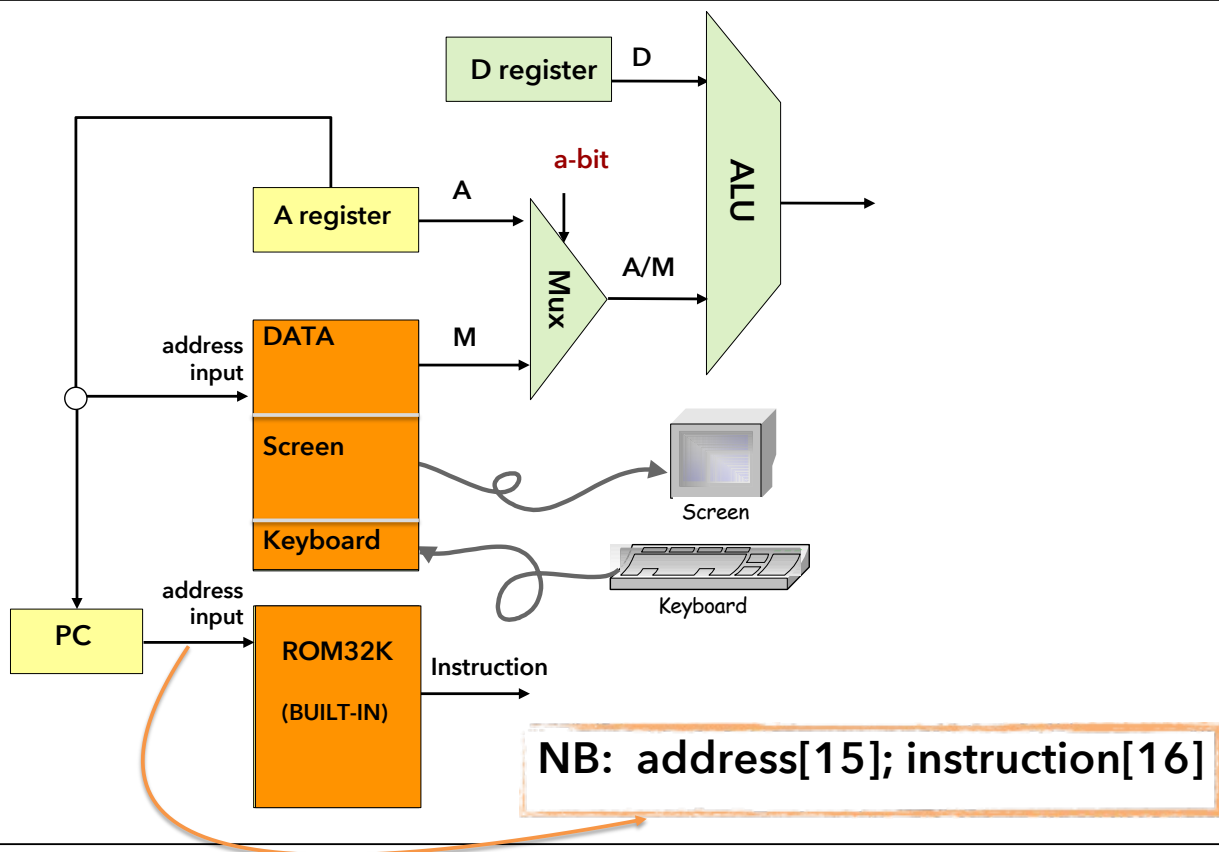
Can be easily built from the chip-set that we built so far in the course

Main parts of the Hack computer:

- ❑ Instruction memory (ROM)
- ❑ Memory (RAM):
 - Data memory
 - Screen (memory map)
 - Keyboard (memory map)
- ❑ CPU
- ❑ Computer (the logic that holds everything together).

7

The Hack computer Architecture



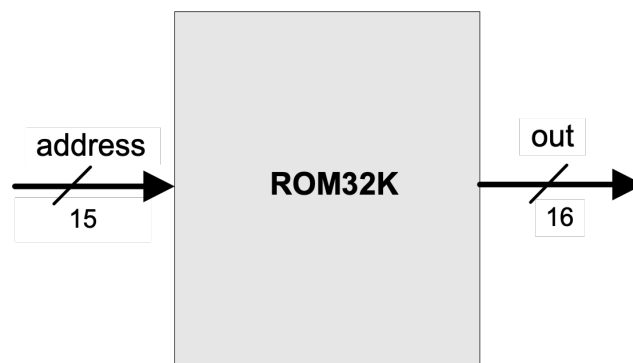
8

Lecture / construction plan

- ➡ ■ Instruction memory
 - Memory:
 - Data memory
 - Screen
 - Keyboard
 - CPU
 - Computer

9

Instruction memory



Function:

- The ROM is pre-loaded with a program written in the Hack machine language
- The ROM chip always emits a 16-bit number:
$$\text{out} = \text{ROM32K}[\text{address}]$$
- This number is interpreted as the current instruction.

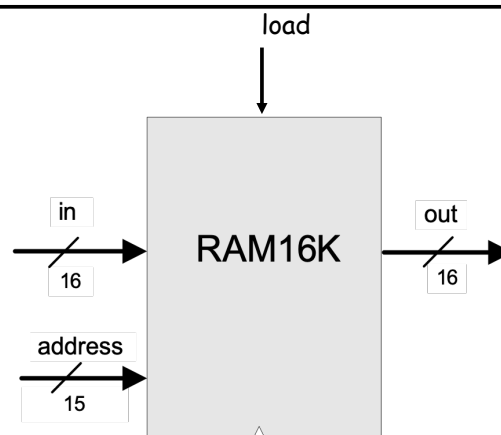
10

Lecture / construction plan

- ☑ ■ Instruction memory
 - Memory:
 - ➡ □ Data memory
 - Screen
 - Keyboard
 - CPU
 - Computer

11

Data memory



Low-level (hardware) read/write logic:

To read $\text{RAM}[k]$: set address to k ,
probe out

To write $\text{RAM}[k]=x$: set address to k ,
set in to x ,
set load to 1,
run the clock

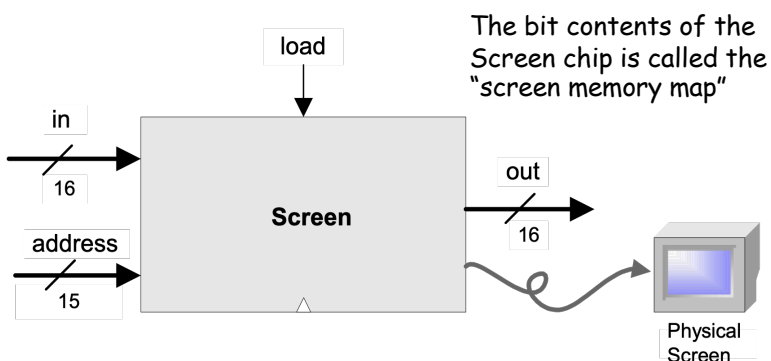
13

Lecture / construction plan

- ☑ ■ Instruction memory
 - Memory:
 - ☑ □ Data memory
 - ➡ □ Screen
 - Keyboard
 - CPU
 - Computer

14

Screen



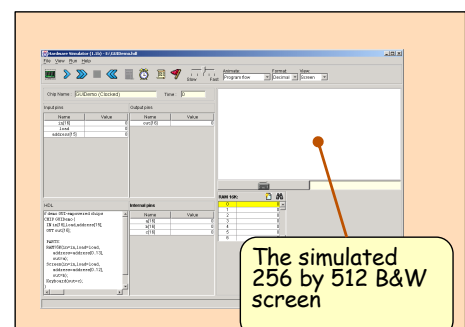
The Screen chip has a basic RAM chip functionality:

- read logic: `out = Screen[address]`
- write logic: `if load then Screen[address] = in`

Side effect:

Continuously refreshes a 256 by 512 black-and-white screen device

Simulated screen:

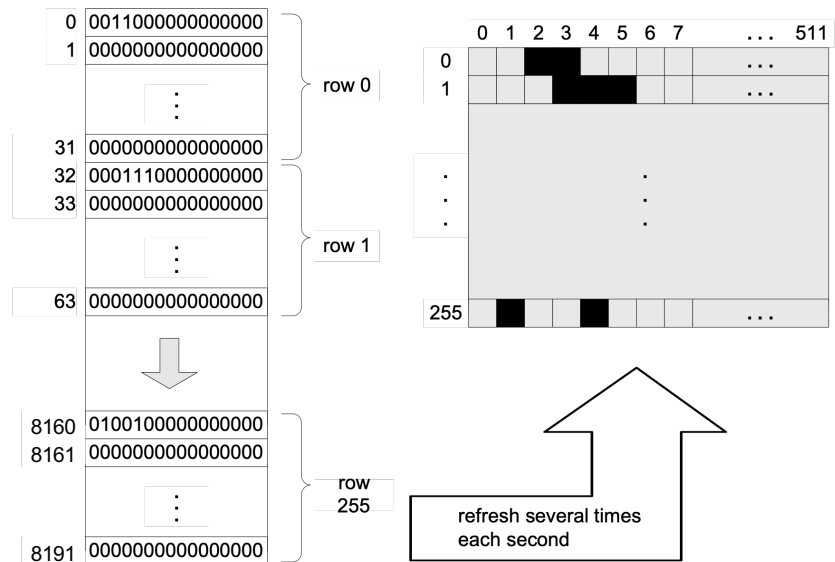
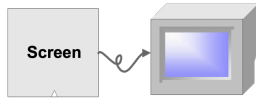


When loaded into the hardware simulator, the built-in Screen.hdl chip opens up a screen window; the simulator then refreshes this window from the screen memory map several times each second.

16

Screen memory map

In the Hack platform, the screen is implemented as an 8K 16-bit RAM chip.



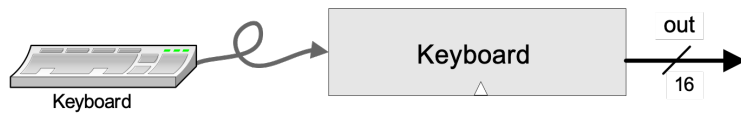
How to set the (row,col) pixel of the screen to black or to white:

- Low-level (machine language): Set the $col\%16$ bit of the word found at $Screen[row*32+col/16]$ to 1 or to 0 ($col/16$ is integer division)

Lecture / construction plan

- ☑ ■ Instruction memory
- Memory:
 - ☑ □ Data memory
 - ☑ □ Screen
- ➡ □ Keyboard
- CPU
- Computer

Keyboard

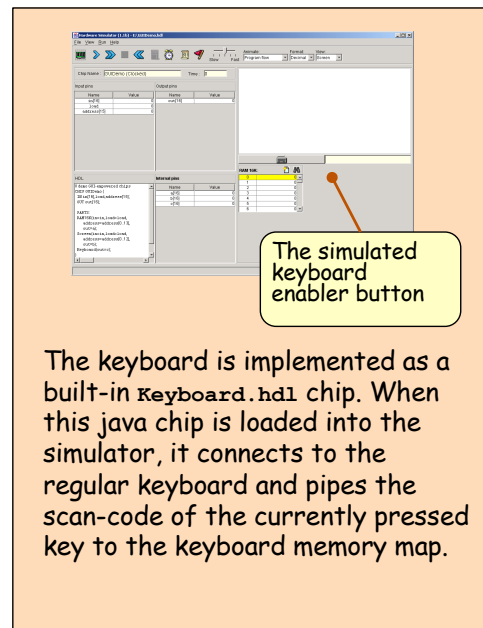


Keyboard chip: a single 16-bit register

Input: scan-code (16-bit value) of the currently pressed key, or 0 if no key is pressed

Output: same

Simulated keyboard:



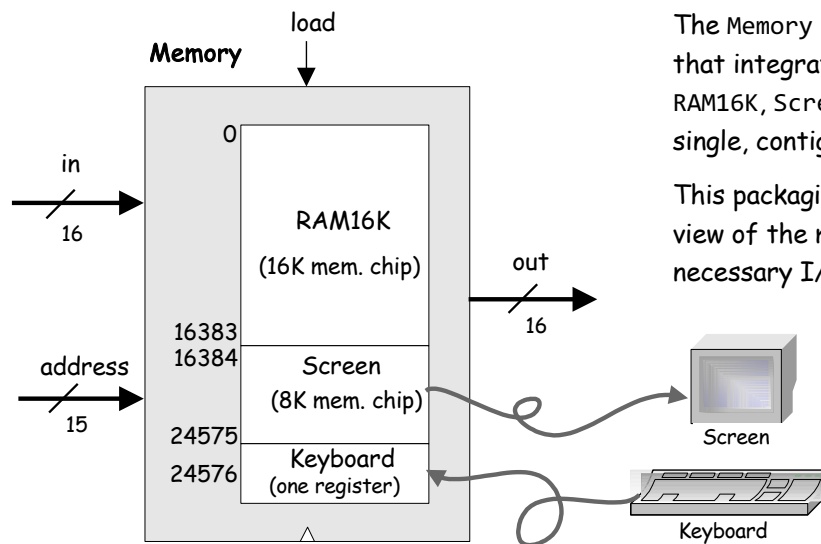
How to read the keyboard:

- Low-level (hardware): probe the contents of the Keyboard chip

Lecture / construction plan

- ☑ ■ Instruction memory
- ➡ ■ Memory:
 - ☑ □ Data memory
 - ☑ □ Screen
 - ☑ □ Keyboard
- CPU
- Computer

Memory: physical implementation



The Memory chip is essentially a package that integrates the three chip-parts RAM16K, Screen, and Keyboard into a single, contiguous address space.

This packaging effects the programmer's view of the memory, as well as the necessary I/O side-effects.

Access logic:

- ☐ Access to any address from 0 to 16,383 results in accessing the RAM16K chip-part
- ☐ Access to any address from 16,384 to 24,575 results in accessing the Screen chip-part
- ☐ Access to address 24,576 results in accessing the keyboard chip-part
- ☐ Access to any other address is invalid.

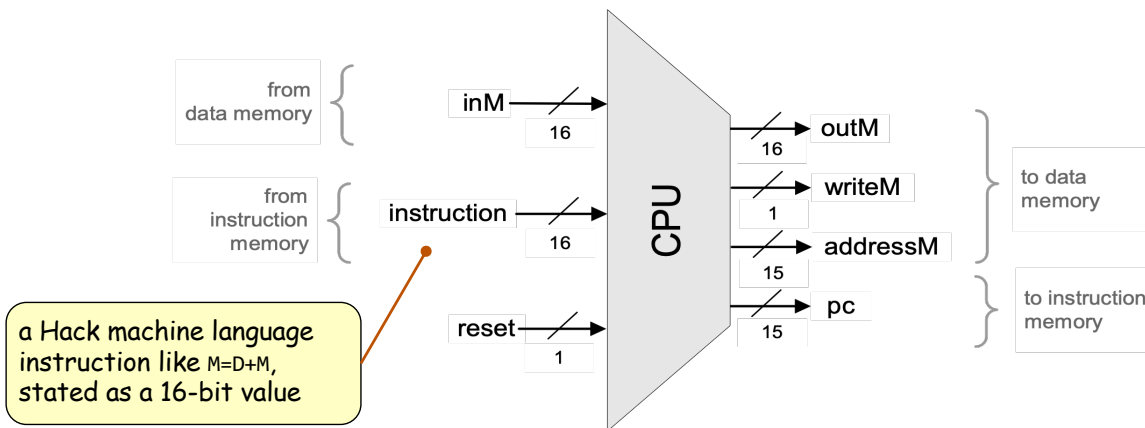
26

Lecture / construction plan

- ☒ ■ Instruction memory
- Memory:
 - ☒ ■ Data memory
 - ☒ ■ Screen
 - ☒ ■ Keyboard
- ➡ ■ CPU
- Computer

27

CPU



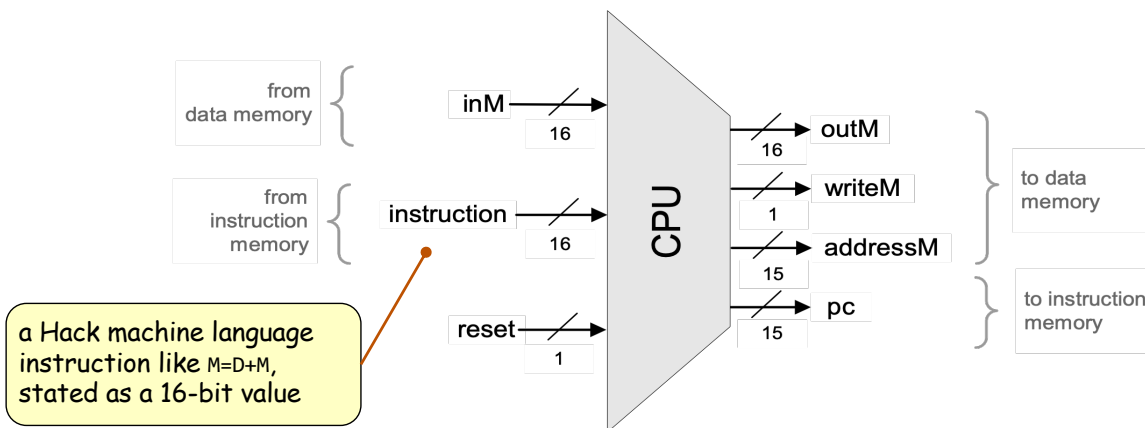
CPU internal components (invisible in this chip diagram): ALU and 3 registers: A, D, PC

CPU execute logic:

The CPU executes the instruction according to the Hack language specification:

- The D and A values, if they appear in the instruction, are read from (or written to) the respective CPU-resident registers
- The M value, if there is one in the instruction's RHS, is read from inM
- If the instruction's LHS includes M, then the ALU output is placed in outM, the value of the CPU-resident A register is placed in addressM, and writeM is asserted.

CPU



CPU internal components (invisible in this chip diagram): ALU and 3 registers: A, D, PC

CPU fetch logic:

Recall that:

1. the instruction may include a jump directive (expressed as non-zero jump bits)
2. the ALU emits two control bits, indicating if the ALU output is zero or less than zero

If reset==0: the CPU uses this information (the jump bits and the ALU control bits) as follows:

If there should be a jump, the PC is set to the value of A; else, PC is set to PC+1

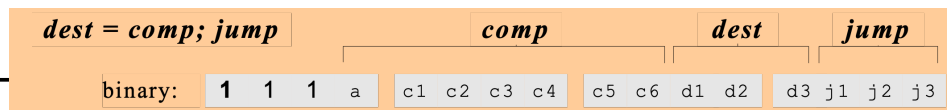
If reset==1: the PC is set to 0. (restarting the computer)

CPU implementation

a1	a2	a3	Mnemonic	Destination (where to store the computed value)
0	0	0	null	The value is not stored anywhere
0	0	1	M	Memory[A] (memory register addressed by A)
0	1	0	D	D register
0	1	1	MD	Memory[A] and D register
1	0	0	A	A register
1	0	1	AM	A register and Memory[A]
1	1	0	AD	A register and D register
1	1	1	AMD	A register, Memory[A], and D register

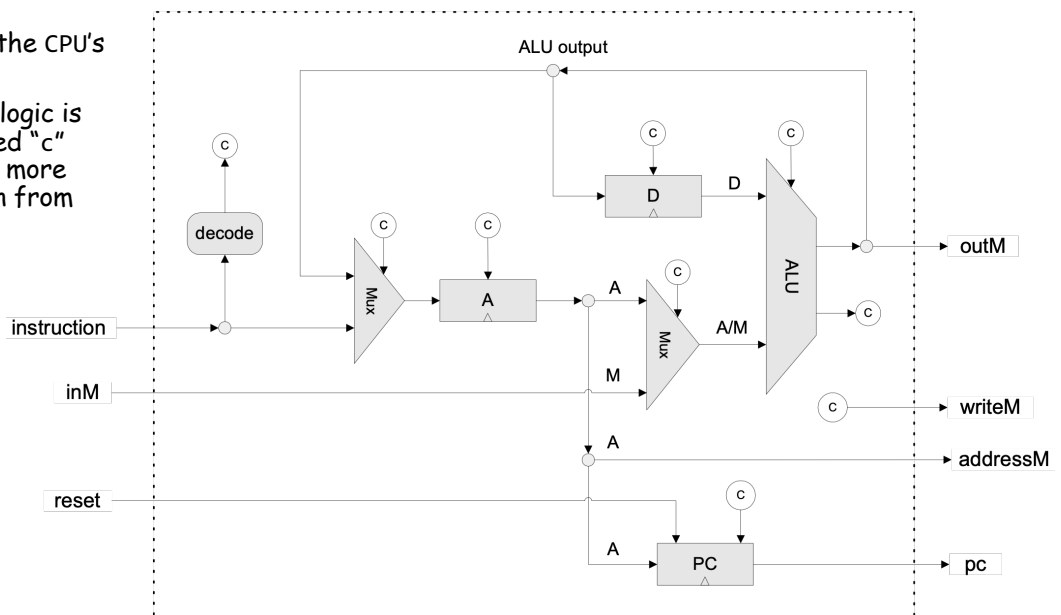
j1 (out < 0)	j2 (out = 0)	j3 (out > 0)	Mnemonic	Effect
0	0	0	null	No jump
0	0	1	JGT	If out > 0 jump
0	1	0	JEQ	If out = 0 jump
0	1	1	JGE	If out ≥ 0 jump
1	0	0	JLT	If out < 0 jump
1	0	1	JNE	If out ≠ 0 jump
1	1	0	JLE	If out ≤ 0 jump
1	1	1	JMP	Jump

CPU implementation



Chip diagram:

- ❑ Includes most of the CPU's execution logic
- ❑ The CPU's control logic is hinted: each circled "c" represents one or more control bits, taken from the instruction
- ❑ The "decode" bar does not represent a chip, but rather indicates that the instruction bits are decoded somehow.



Cycle:

- Execute
- Fetch

Execute logic:

- ❑ Decode
- ❑ Execute

Fetch logic:

```
If there should be a jump,  
    set PC to A  
else set PC to PC+1
```

Resetting the computer:

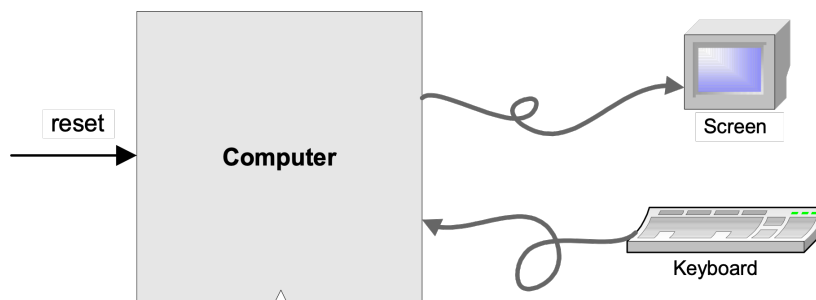
Set reset to 1,
then set it to 0.

Lecture / construction plan

- ☑ ■ Instruction memory
- ☑ ■ Memory:
 - Data memory
 - Screen
 - Keyboard
- ☑ ■ CPU
- ➡ ■ Computer

34

Computer-on-a-chip interface

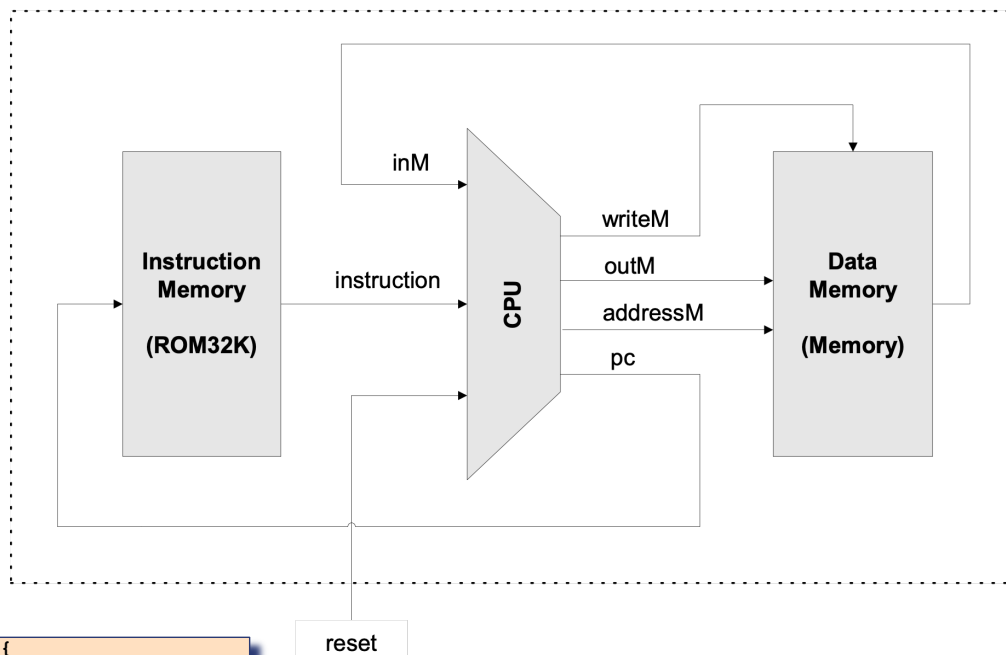


```
Chip Name: Computer // Topmost chip in the Hack platform
Input:      reset
Function:   When reset is 0, the program stored in the
            computer's ROM executes. When reset is 1, the
            execution of the program restarts. Thus, to start a
            program's execution, reset must be pushed "up" (1)
            and "down" (0).

            From this point onward the user is at the mercy of
            the software. In particular, depending on the
            program's code, the screen may show some output and
            the user may be able to interact with the computer
            via the keyboard.
```

35

Computer-on-a-chip implementation



```
CHIP Computer {
    IN reset;
    PARTS:
        // implementation missing
}
```

Implementation:

Simple, the chip-parts do all the hard work.

36

Project 5: Computer Architecture

Chip (HDL)	Description	Testing
Memory.hdl	Entire RAM address space	Test this chip using Memory.tst and Memory.cmp
CPU.hdl	The Hack CPU	Recommended test files: CPU.tst and CPU.cmp . Alternative test files (less thorough but do not require using the built-in DRegister): CPU-external.tst and CPU-external.cmp .
Computer.hdl	The platform's top-most chip	Test by running some Hack programs on the constructed chip. See more instructions below.

- The computer platform should be capable of executing programs written in the Hack machine language
- Demonstrate this capability by having your Computer chip run the three test programs given below:

Program	Comments
Add.hack	Adds up the two constants 2 and 3 and writes the result in RAM[0]. Recommended test: ComputerAdd.tst and ComputerAdd.cmp . Alternative test (less thorough but only requires usage of the built-in RAM16K): ComputerAdd-external.tst and ComputerAdd-external.cmp .
Max.hack	Computes the maximum of RAM[0] and RAM[1] and writes the result in RAM[2]. Recommended test: ComputerMax.tst and ComputerMax.cmp . Alternative test (less thorough but only requires usage of the built-in RAM16K): ComputerMax-external.tst and ComputerMax-external.cmp .
Rect.hack	Draws a rectangle of width 16 pixels and length RAM[0] at the top left of the screen. Recommended test: ComputerRect.tst and ComputerRect.cmp . Alternative test (less thorough but does not require usage of any built-in chips): ComputerRect-external.tst and ComputerRect-external.cmp .

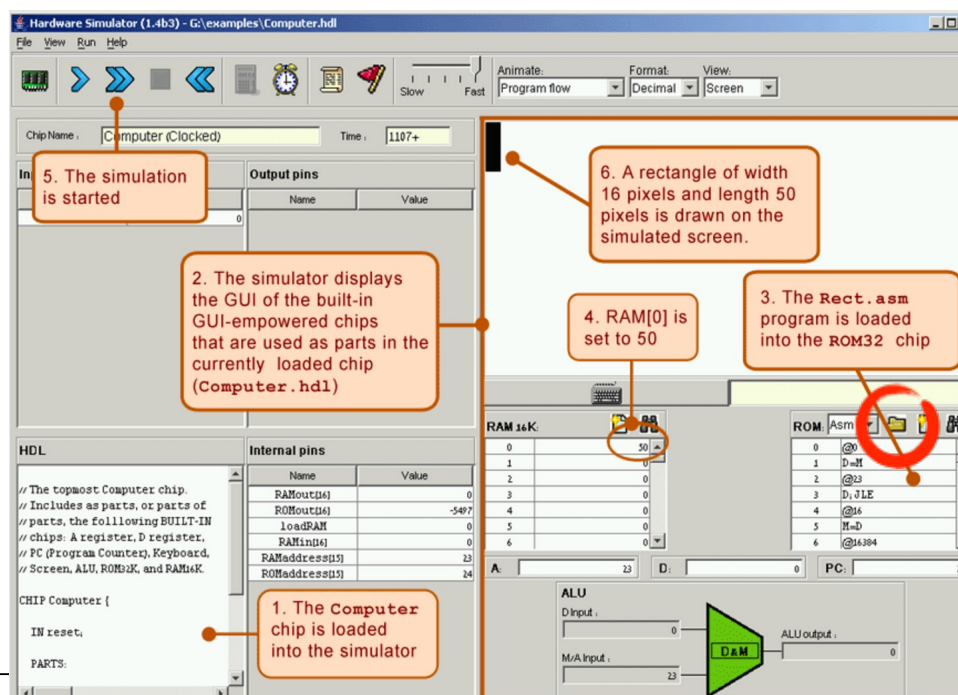
Project 5 - Testing notes

- The provided .tst files include ***-external.tst** versions
 - They should **NOT** be employed if built-in versions of registers and PC are adopted to realize the CPU (as prescribed)
 - Standard .tst files should be employed for verifying the realized .hdl files

38

Project 5 - Testing notes

- The Computer chip can be tested also by manually loading the ROM



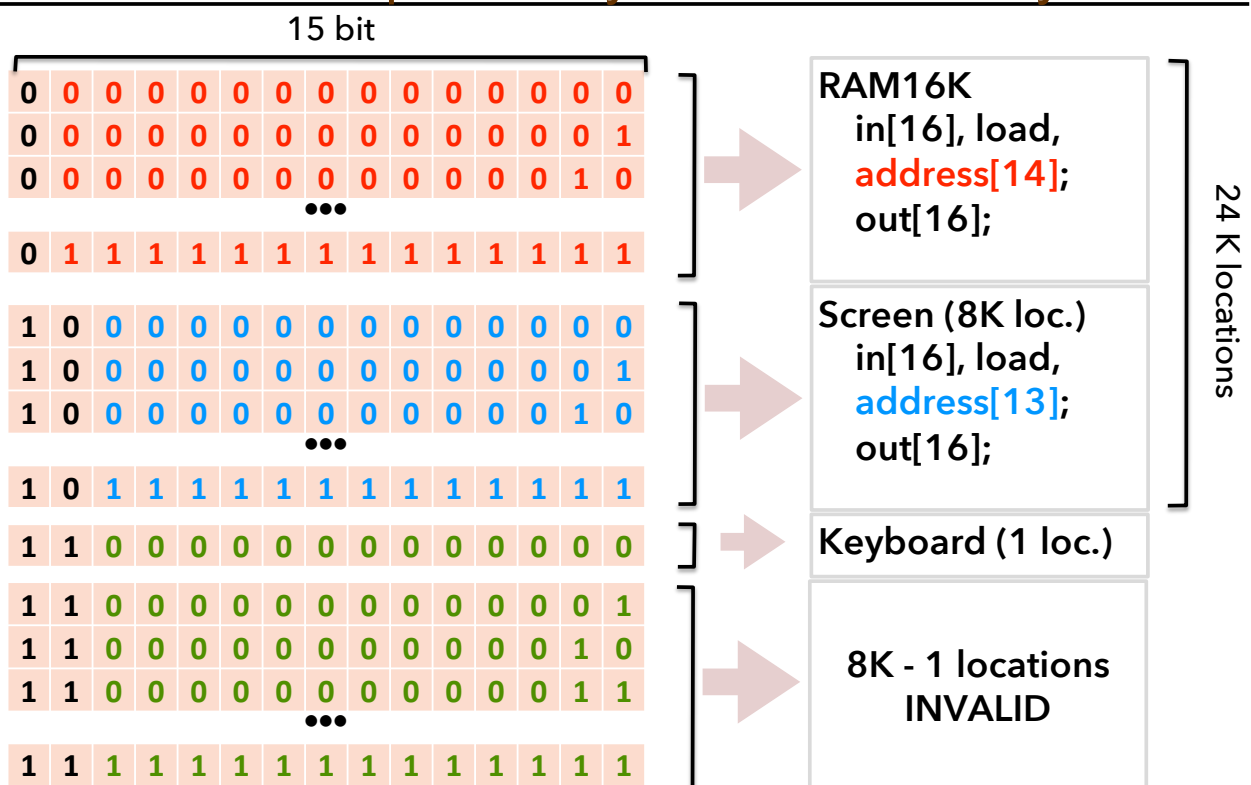
39

With a little help from my friends - Memory

- Memory is composed by
 - RAM16K (addresses 0000h-3FFFh)
 - Screen (addresses 4000h-5FFFh) - built-in provided
 - Keyboard (address 6000h) - built-in provided
- Referring to addresses greater that 6000h is invalid
 - actually those addresses can be simply ignored
 - keyboard is activated from a certain address on
- The **load signal** must be properly directed
 - When RAM is referred, Screen should not be wrote in
 - When Screen is referred, RAM should not be wrote in

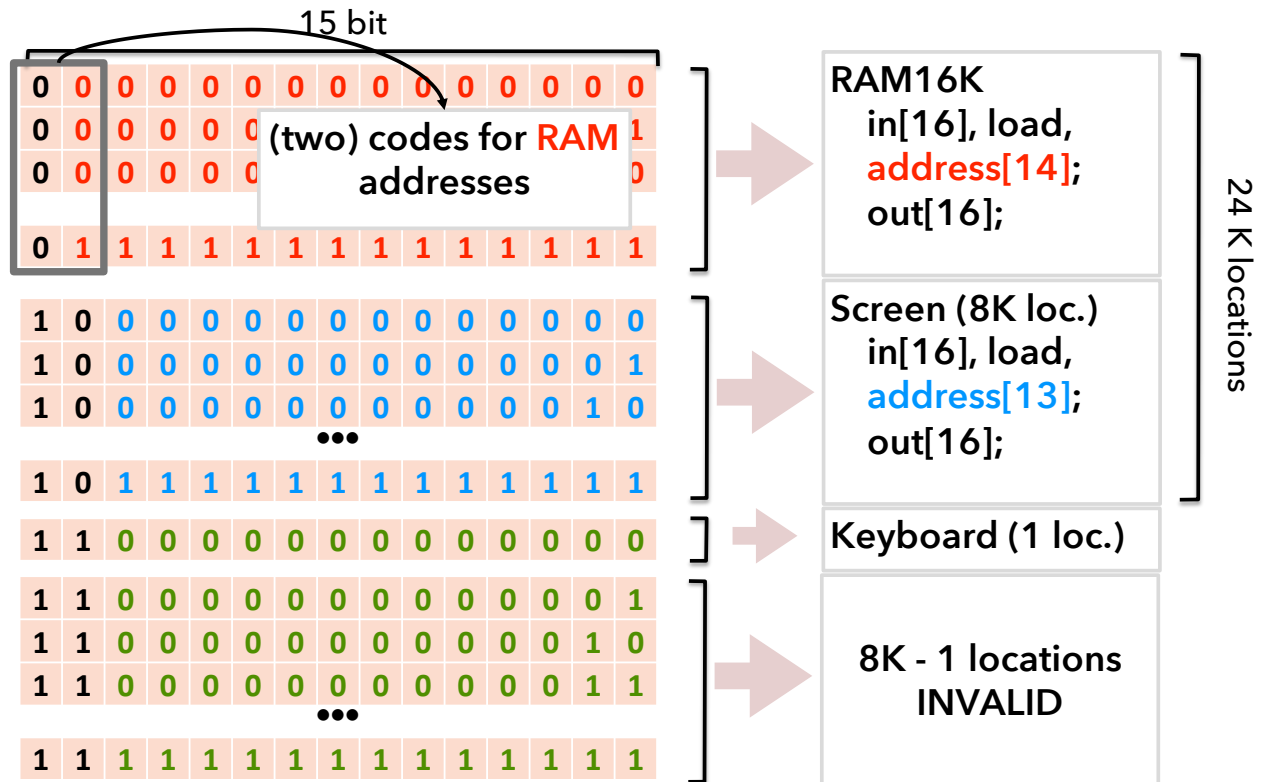
40

With a little help from my friends - Memory



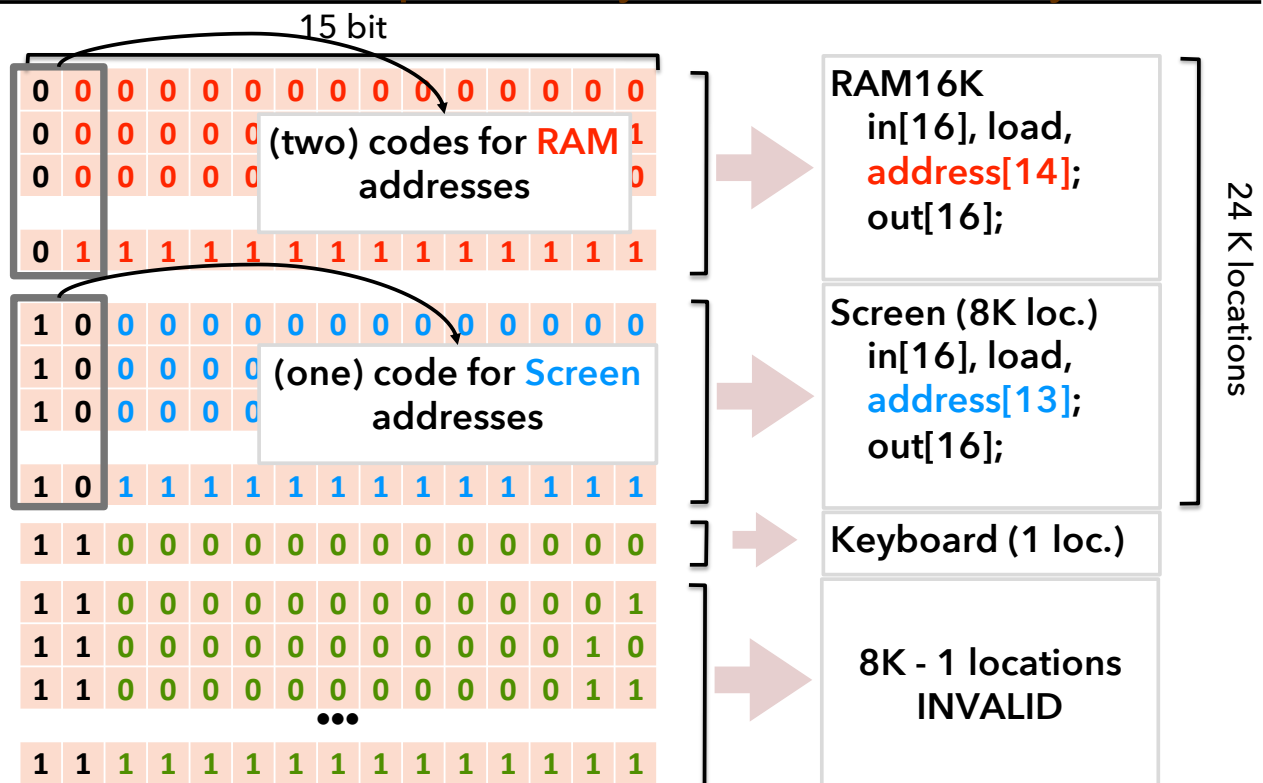
41

With a little help from my friends - Memory



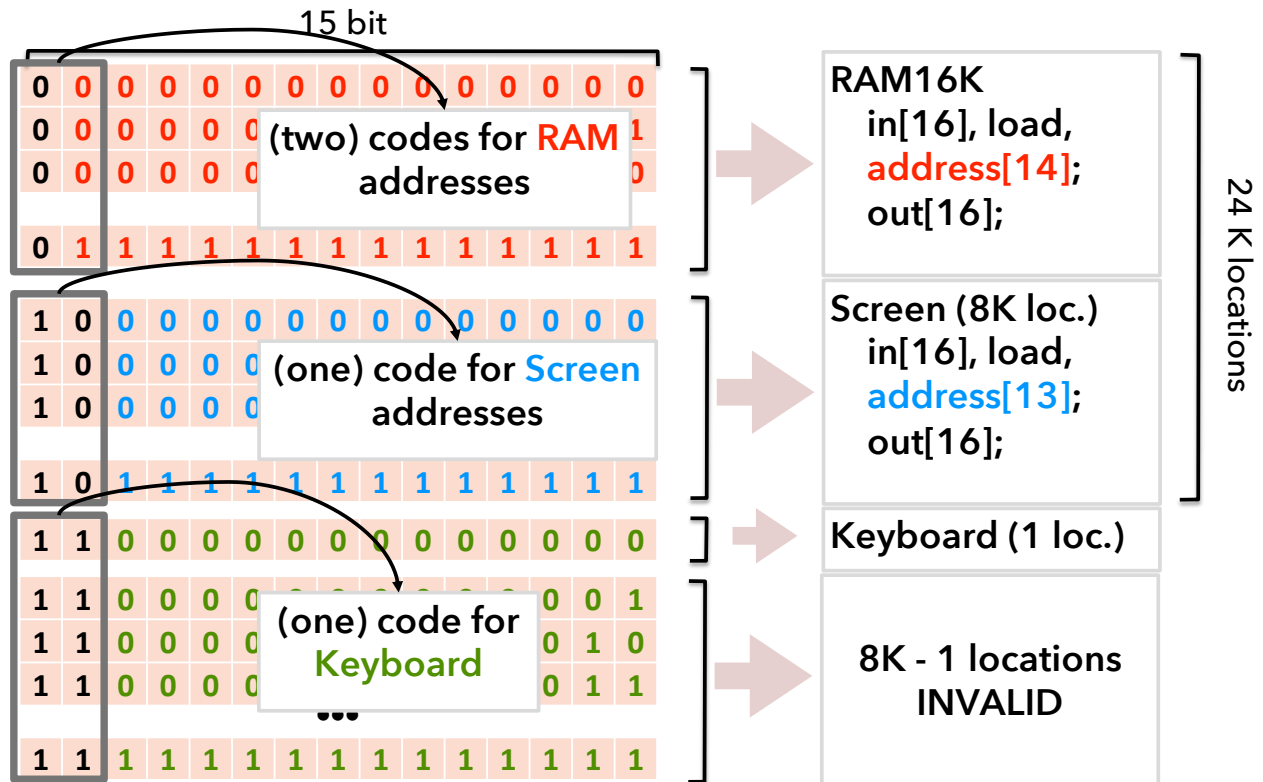
42

With a little help from my friends - Memory



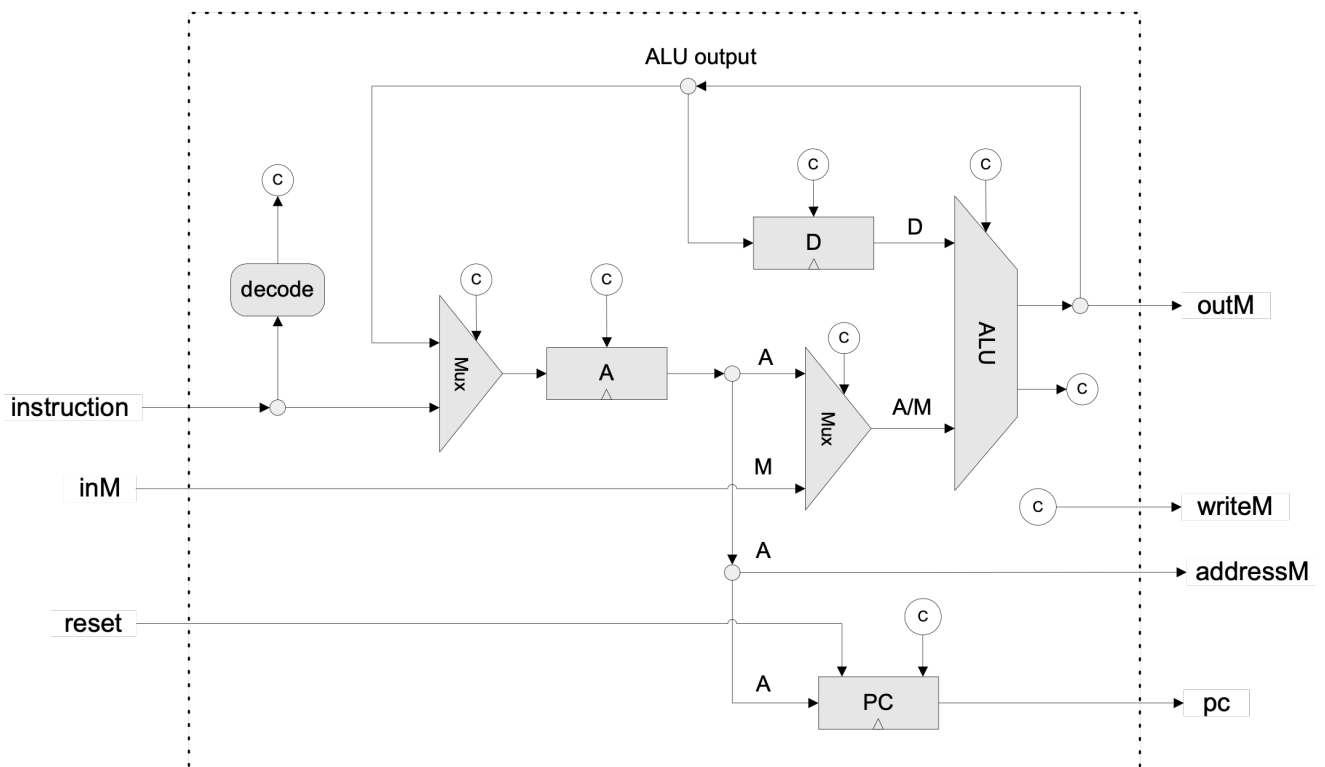
43

With a little help from my friends - Memory



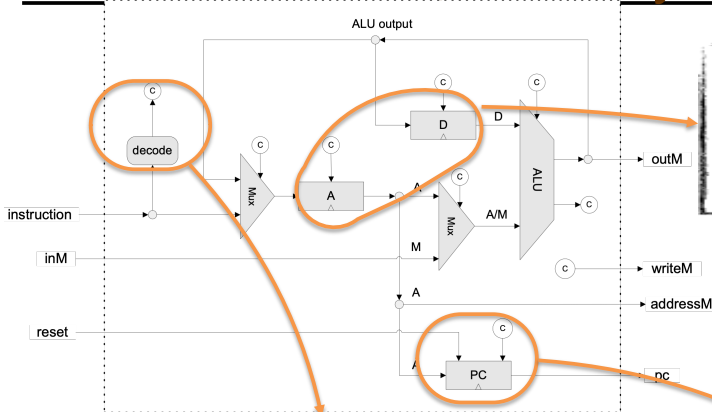
44

With a little help from my friends - CPU



45

With a little help from my friends - CPU



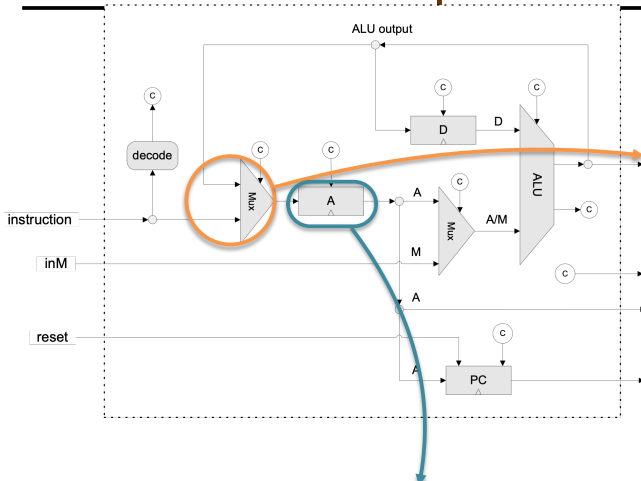
- A and D are registers
 - ARegister and DRegister
built-in provided

- Decoding instructions
 - A-instr vs C-instr
 - D and M are loaded only with C-instr
 - A is loaded with both A-instr and C-instr
 - Jump only with C-instr

- Jumping is loading a specific value in PC
- Deciding to jump:
 - bits j1, j2, j3 (instr)
 - bits zr, ng (ALU)
- Jump only with C-instr

46

With a little help from my friends - CPU

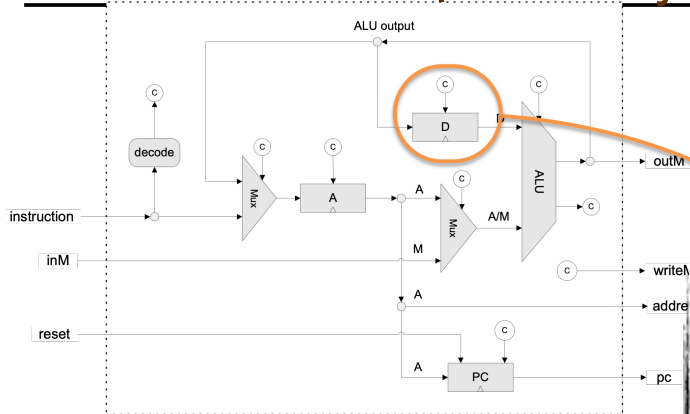


- Two different inputs for A
 - from instr (A is set to a value)
 - or
 - from ALU (A is a destination)
- Governed by the type of the instruction

- At least one of two cases must occur to load A:
 - A-instruction
 - A is a destination

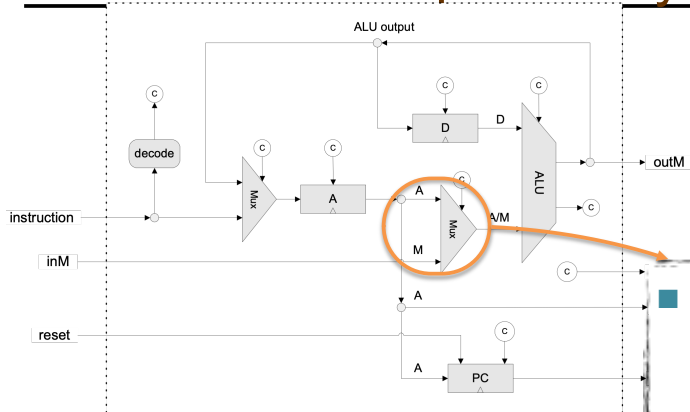
47

With a little help from my friends - CPU



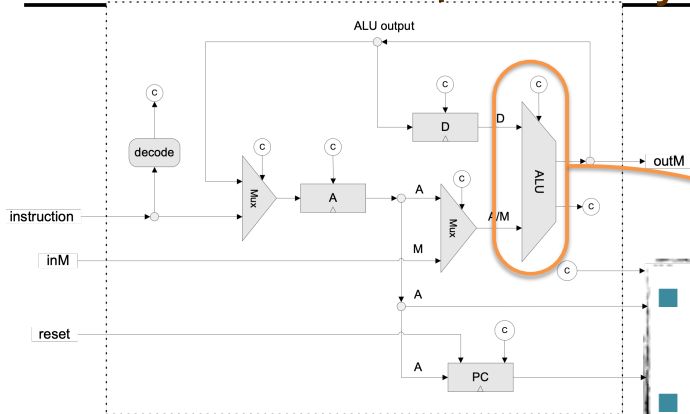
- D register is loaded when
 - C-instruction
 - and
 - D is destination

With a little help from my friends - CPU



- The other ALU operand can be either A or M
 - this information is coded into the C-instruction
 - see the comp table

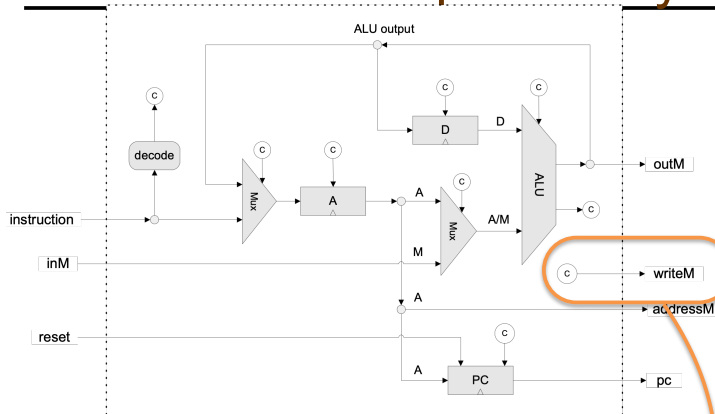
With a little help from my friends - CPU



- The ALU receives two operands
 - plus
- All of its control bits
- See the **comp** table

50

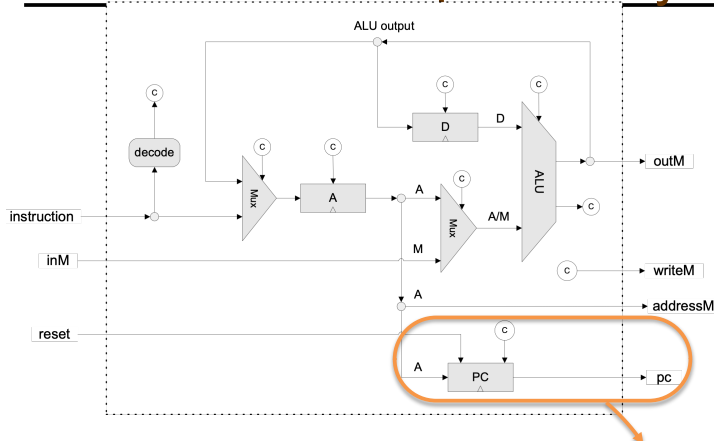
With a little help from my friends - CPU



- The writeM line is asserted when
 - C-instruction
 - and
 - M is destination

51

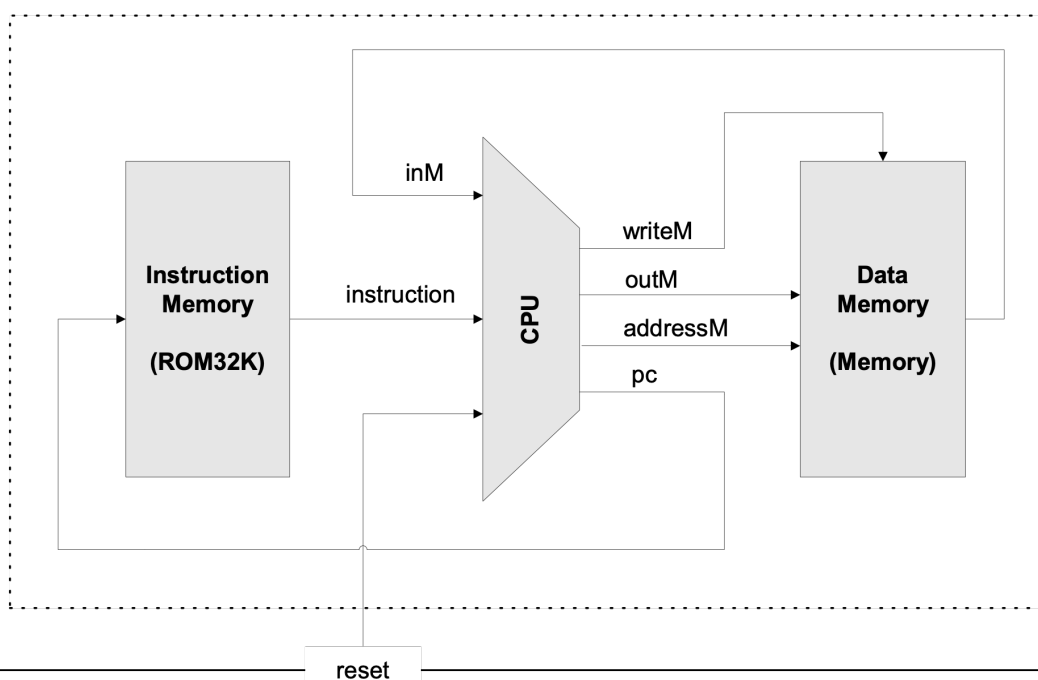
With a little help from my friends - CPU



- PC should be **always** incremented, but
 - increment is unaffected when a jump/reset is executed
- The "jump if < 0", "jump if = 0", "jump if > 0" cases must be identified
 - cases must be identified combining the information from **jump bits** (from instruction) and **control bits** (from ALU)
 - jumps **are NOT effective** in case of A-instructions

With a little help from my friends - Computer

- Implementation is **trivial**: just compose the figure
 - ROM32K is built-in provided



With a little help from my friends - delivery instructions

■ Please adhere to the following notation for delivering the .hdl files

- preserve the directory structure while working on your project files
- submit one zip file

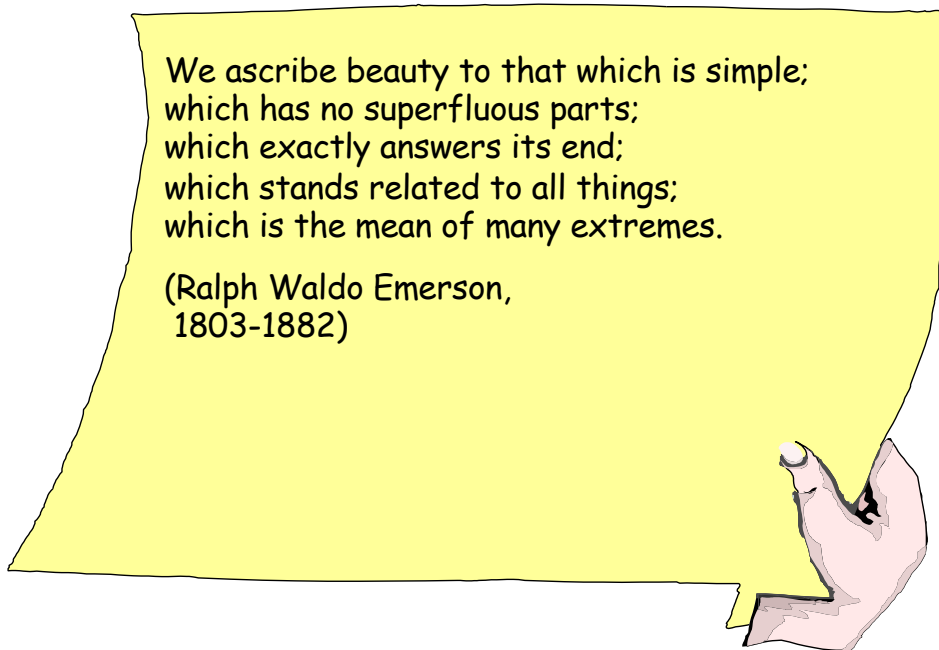
_P05CognomeNome.zip (containing Memory.hdl, CPU.hdl, Computer.hdl)

The zip file must contain immediately the .hdl files, with no additional file or directory

The spirit of things

We ascribe beauty to that which is simple;
which has no superfluous parts;
which exactly answers its end;
which stands related to all things;
which is the mean of many extremes.

(Ralph Waldo Emerson,
1803-1882)



Lecture plan



"That's
all
folks!"

- ☑ ■ Instruction memory
- ☑ ■ Memory:
 - ☑ □ Data memory
 - ☑ □ Screen
 - ☑ □ Keyboard
- ☑ ■ CPU
- ☑ ■ Computer



"Ya, right,
but what about
the **software**?"